



TECHNISCHE DOKUMENTATION

Angeli

Swiss League Fantasy

Vollständige Beschreibung von Aufbau, Spielregeln, Datenmodell und Betrieb — so detailliert, dass sich das Spiel daraus von Grund auf nachbauen liesse. Inklusive der Fehler, die beim ersten Bau gemacht wurden, und wie sie vermieden werden.

Saison	Stand	Umfang	Live
2026/27, Schweizer Liga	Juli 2026	~8'350 Zeilen, 15 Migrationen	angeli-fantasy.org

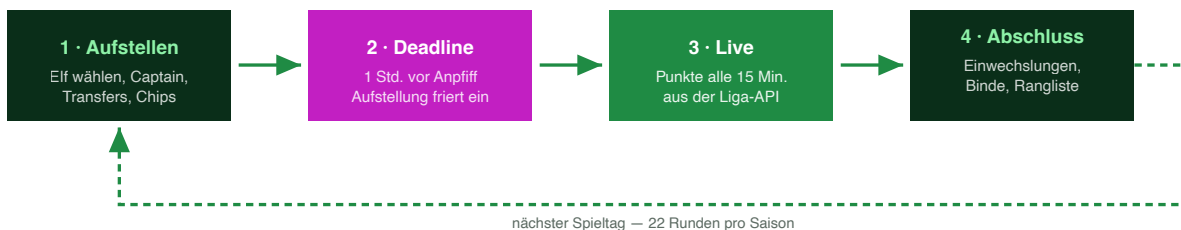
Inhalt

1	Was das Spiel ist	Grundidee, Spielschleife
2	Systemarchitektur	Dienste, Kosten, Datenfluss
3	Datenmodell	16 Tabellen, Beziehungen
4	Spielregeln im Detail	Kader, Punkte, Transfers, Chips
5	Der Spieltag-Lebenszyklus	Deadline, Live, Abschluss
6	Automatiken	Rollover, Einwechslung, Binde
7	Sicherheitsmodell	RLS, Schreibpfade, Angriffe
8	Administration	Werkzeuge, Überwachung
9	Fallstricke	Gefundene Fehler, Lehren
10	Nachbau-Anleitung	Reihenfolge, Prüfpunkte

1 Was das Spiel ist

Angeli ist ein Fantasy-Football-Spiel für die Schweizer Liga, gebaut als Ersatz für die eingestellte 20min-Fantasy-Liga. Mitglieder stellen aus echten Ligaspielern ein Team zusammen und sammeln Punkte danach, wie diese Spieler in echten Partien abschneiden. Das Vorbild ist Fantasy Premier League.

Die Spielschleife



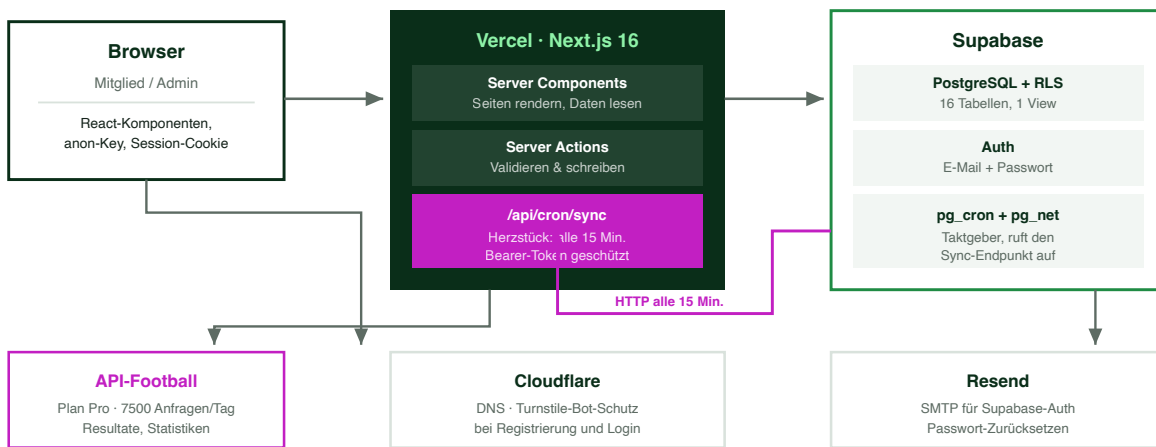
Die vier Phasen jeder Runde. Nur Phase 1 erfordert eine Handlung des Mitglieds; 2 bis 4 laufen vollständig automatisch.

Bewusste Designentscheidungen

ENTSCHEIDUNG	BEGRÜNDUNG
Kein Geld im Spiel	Keine Einsätze, keine Geldpreise — hält das Projekt ausserhalb des Schweizer Geldspielgesetzes.
Punkte nie gespeichert, immer berechnet	Die Rangliste ist eine Datenbank-View über die Rohdaten. Es gibt keinen Punktestand, der von der Realität abweichen könnte.
Budget als Differenz, nicht als Konto	Restbudget = Obergrenze minus Summe der aktuellen Spielerpreise. Kein Kontostand, der inkonsistent werden kann.
Alle Schreibvorgänge über den Server	Der Browser darf Spielstandtabellen nicht beschreiben. Sonst liesse sich jede Validierung per direktem API-Aufruf umgehen.
Späteinstieg ausdrücklich vorgesehen	Erste Runde transferfrei, zwischen Runde 3 und 5 zusätzlich Punkte-Gutschrift in Höhe des Durchschnitts.

2 Systemarchitektur

Fünf Dienste, alle im Gratis- oder Kleinsttarif. Die Gesamtkosten liegen bei etwa 20–30 CHF pro Monat, praktisch vollständig für den Datenzugang.



Wichtig für den Nachbau

Der Dienstschlüssel (service role) und der API-Schlüssel liegen ausschliesslich auf dem Server. Der Browser kennt nur den öffentlichen anon-Key, der durch Row Level Security streng begrenzt ist. Der Sync läuft serverseitig — deshalb greifen Domain- oder CORS-Beschränkungen von Daten-APIs hier gar nicht erst.

Gesamtarchitektur. Der einzige bewegliche Teil im Dauerbetrieb ist der Sync-Endpoint; alles andere reagiert nur auf Anfragen.

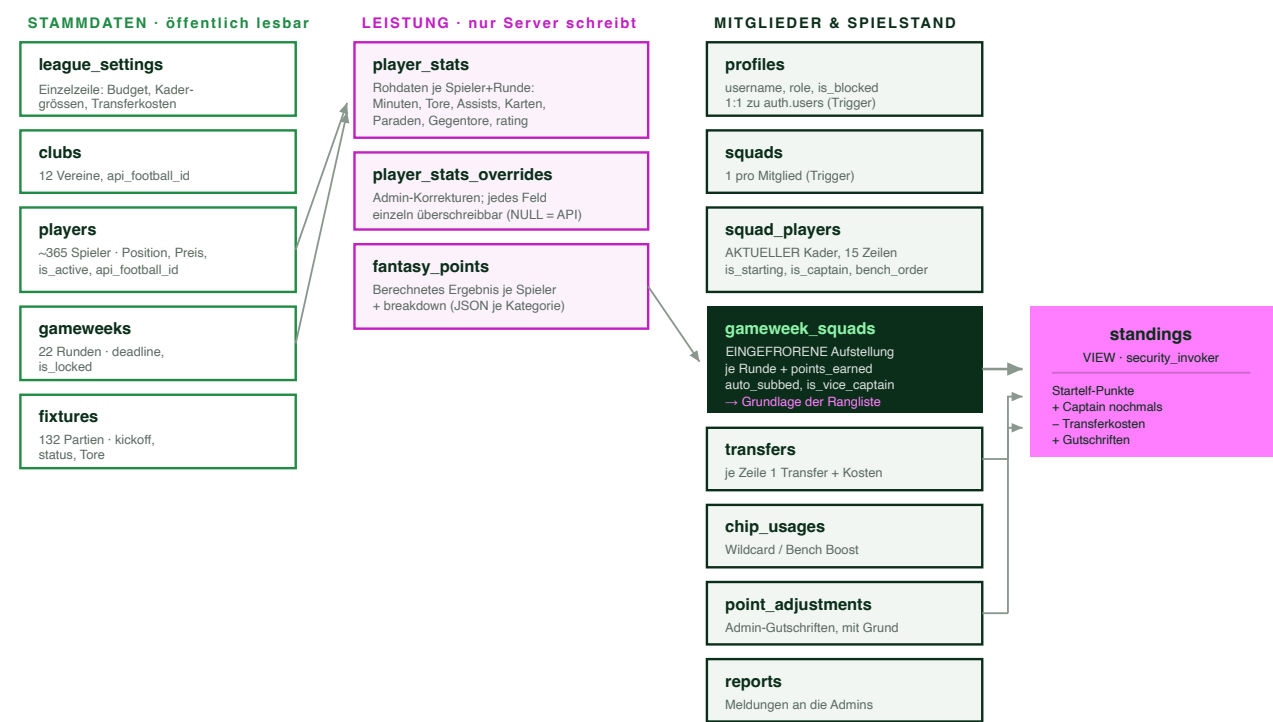
Warum pg_cron statt Vercel-Cron

KOSTENFALLE

Vercels Gratistarif erlaubt nur **einen Cron-Lauf pro Tag**. Für Live-Punkte während laufender Spiele ist das unbrauchbar. Lösung: `pg_cron` in Supabase übernimmt den 15-Minuten-Takt und ruft den Endpoint per `pg_net` über HTTP auf. Der Vercel-Cron bleibt als täglicher Rückfall bestehen. Kosten dafür: null.

3 Datenmodell

16 Tabellen und eine View. Die Aufteilung folgt einem Grundsatz: **Stammdaten** (Clubs, Spieler, Spieltage) sind für alle lesbar, **Spielstände** (Kader, Aufstellungen, Transfers) gehören je einem Mitglied und sind für den Browser schreibgeschützt.



Der entscheidende Trick: **squad_players** ist der lebende Kader, **gameweek_squads** die eingefrorene Kopie je Runde.
Nur Letztere zählt für Punkte — dadurch ändert ein Transfer heute nie das Ergebnis von letzter Woche.

Datenmodell mit Schreibrechten. Grün = für alle lesbar, Magenta = nur Server, Grau = mitgliedsgebunden. Die Rangliste ist eine reine Sicht auf die Rohdaten.

Die zentrale Trennung: lebender Kader vs. eingefrorene Aufstellung

Dies ist die wichtigste Entwurfsentscheidung des ganzen Systems und der Punkt, an dem ein Nachbau am ehesten scheitert.

	SQUAD_PLAYERS	GAMEWEEK_SQUADS
Inhalt	Der Kader, wie er <i>jetzt</i> aussieht	Kopie der Aufstellung <i>je Runde</i>
Zeilen	Genau 15 pro Mitglied	15 pro Mitglied <i>und</i> Runde
Wird geändert	Bei jedem Speichern	Beim Speichern für die offene Runde; danach nur noch durch Automaten
Zählt für Punkte	Nein	Ja — ausschliesslich
Sichtbar für andere	Nie	Erst nach Ablauf der Deadline

FALLSTRICK BEIM NACHBAU

Es ist verlockend, die Aufstellung einer vergangenen Runde aus `squad_players` zu rekonstruieren. Das ist falsch: Sobald jemand für die nächste Runde umbaut, verändert sich dieser Datensatz. Alles, was rückwirkend zählt, muss zum Zeitpunkt des Deadline-Ablaufs kopiert worden sein.

Tabellen im Einzelnen

TABELLE	ZWECK	BESONDERHEIT
---------	-------	--------------

league_settings	Alle Stellschrauben in einer Zeile	Auch Sync-Heartbeat und Ankündigungs-Banner
clubs	12 Vereine	short_name ist zugleich Dateiname des Trikot-Bildes
players	Kaderspieler aller Clubs	is_active statt Löschen — Abgänge bleiben in alten Aufstellungen gültig
gameweeks	22 Runden mit Deadline	is_locked wird automatisch gesetzt, ist aber vom Admin überschreibbar
fixtures	132 Partien	status aus der API steuert Live-Erkennung
profiles	Mitgliedsdaten	Per Trigger aus auth.users erzeugt; enthält bewusst keine E-Mail
squads	Ein Kader je Mitglied	Ebenfalls per Trigger erzeugt
player_stats	Rohdaten aus der API	Eindeutig je (Spieler, Runde); wird bei jedem Sync überschrieben
fantasy_points	Errechnete Punkte	breakdown als JSON — Grundlage des Spieler-Popups
transfers	Eine Zeile je Transfer	Guthaben wird daraus abgeleitet , nicht in einem Zähler geführt

4 Spielregeln im Detail

4.1 Kader und Budget

REGEL	WERT
Kadergrösse	15 Spieler — 2 Torhüter, 5 Verteidiger, 5 Mittelfeldspieler, 3 Stürmer
Startelf	11 Spieler , die übrigen 4 auf der Bank
Club-Limit	Höchstens 3 Spieler desselben Clubs — geprüft von Oberfläche und Server
Budget	100 Mio. als Basis
Preisspanne	4.0 – 10.0, in 0.5er-Schritten
Formation	Genau 1 TW; mind. 3 DEF, 3 MID, 1 ST; höchstens 5 DEF, 5 MID, 3 ST

BUDGET-BESONDERHEIT

Das Restbudget ist **keine gespeicherte Zahl**, sondern die Differenz zwischen Obergrenze und der Summe der aktuellen Spielerpreise. Die persönliche Obergrenze ist $\max(100, \text{Wert des gespeicherten Teams})$: Steigen Preise gehaltener Spieler, wächst der Spielraum mit — niemand wird durch eine Preisanpassung handlungsunfähig. Mehr Wert einkaufen als vorhanden geht dennoch nicht.

4.2 Punktetabelle

EREIGNIS	PUNKTE	BEDINGUNG
Einsatz bis 59 Minuten	+1	—
Einsatz ab 60 Minuten	+2	—

Tor eines Torhüters oder Verteidigers	+6	je Tor
Tor eines Mittelfeldspielers	+5	je Tor
Tor eines Stürmers	+4	je Tor
Assist	+3	je Assist, alle Positionen
Ohne Gegentor (TW, DEF)	+4	nur ab 60 Minuten
Je 2 Paraden eines Torhüters	+1	abgerundet
Gehaltener Elfmeter	+5	—
Je 2 Gegentore (TW, DEF)	-1	nur ab 60 Minuten , abgerundet
Gelbe Karte	-1	—
Verursachter Elfmeter	-2	—
Eigentor	-2	API liefert dies nicht — Admin trägt nach
Rote Karte	-3	—

DIE 60-MINUTEN-SCHWELLE BEIM GEGENTOR-ABZUG

Beim Vorbild zählen nur Gegentore, die fallen, *während* ein Spieler auf dem Platz steht. Die Datenquelle liefert das für Feldspieler nicht — sie kennt nur die Team-Gegentore. Ohne Schwelle kassierte ein in der 73. Minute eingewechselter Verteidiger den vollen Abzug für Tore, die vor seiner Einwechslung fielen; sein Einsatzpunkt wurde dadurch regelmässig auf null aufgeessen. Die Kopplung an dieselbe 60-Minuten-Schwelle wie beim Zu-Null löst das sauber und ist erklärbar.

4.3 Captain und Vize-Captain

- Der **Captain** bringt die doppelte Punktzahl — **auch im Minus**.
- Spielt er **keine Minute**, übernimmt der **Vize-Captain** die Verdopplung — sofern dieser selbst gespielt hat.
- Beide müssen in der Startelf stehen und dürfen nicht dieselbe Person sein.

4.4 Transfers

- **1 Gratis-Transfer** pro Spieltag; jeder weitere kostet **4 Punkte**.
- Nicht genutzte Transfers **sammeln sich an**, maximal 5.
- In der **ersten eigenen Runde** sind Transfers unbegrenzt und gratis.

WARUM DAS GUTHABEN NICHT GEZÄHLT, SONDERN HERGELEITET WIRD

Ein Zählerfeld `free_transfers_remaining` war die ursprüngliche Lösung — und ein Fehler: Es wurde bei jedem Speichern zurückgesetzt, wodurch jeder Transfer gratis war, solange man einzeln speicherte. Heute wird das Guthaben aus der Transferhistorie berechnet:

$$\text{guthaben}(n+1) = \min(\text{cap}, \max(0, \text{guthaben}(n) - \text{genutzt}(n)) + \text{zuwachs})$$

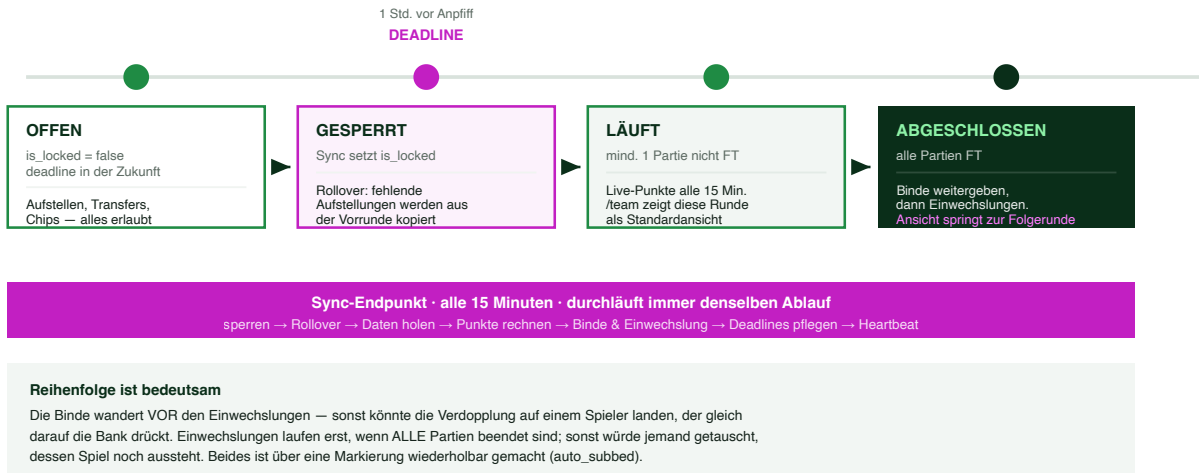
Gebuchte Transfers lassen sich nicht überschreiben, ein Zählerfeld schon.

4.5 Chips

CHIP	WIRKUNG	VERFÜGBARKEIT
 Wildcard	Beliebig viele Transfers in dieser Runde, ohne Punktabzug	Je einmal pro Saison . Bis zur Deadline zurücknehmbar, danach verbraucht.
 Bench Boost	Auch die Punkte der vier Bankspieler zählen	

5 Der Spieltag-Lebenszyklus

Jede Runde durchläuft vier Zustände. Der Übergang geschieht ausschliesslich durch den Sync-Endpunkt — es gibt keinen manuellen Schritt, der zwingend nötig wäre.



Zustandsübergänge einer Runde. Alle Übergänge sind zeitgesteuert und laufen ohne Zutun — der Admin kann sie überschreiben, muss aber nicht eingreifen.

Was der Sync-Endpunkt genau tut

#	SCHRITT	BESCHREIBUNG
1	Runden sperren	Alle Spieltage mit abgelaufener Deadline erhalten <code>is_locked = true</code> .
2	Rollover	Kader ohne Aufstellung für diese Runde erhalten eine Kopie ihrer letzten Aufstellung — aber nur, wenn sie an einer früheren Runde teilgenommen haben.
3	Partien holen	Resultate und Status der aktuellen Runde von der API; <code>cache: "no-store"</code> , sonst kämen veraltete Daten.
4	Statistiken holen	Je Partie mit Daten die Spielerstatistiken. Gegentore werden aus dem Resultat abgeleitet , nicht aus dem Spielfeld.
5	Punkte rechnen	Für jeden berührten Spieler: Rohdaten + Admin-Korrekturen → Punkte + Aufschlüsselung.
6	Binde & Einwechslung	Nur wenn alle Partien beendet sind.
7	Deadlines pflegen	Für offene Runden: Deadline = erster Anpfiff minus 1 Stunde. Spielverschiebungen korrigieren sich damit selbst.
8	Heartbeat	Zeitstempel und Kurzbericht in <code>league_settings</code> — Grundlage der Admin-Ampel.

6 Die drei Automaten

6.1 Rollover — Aufstellung läuft weiter

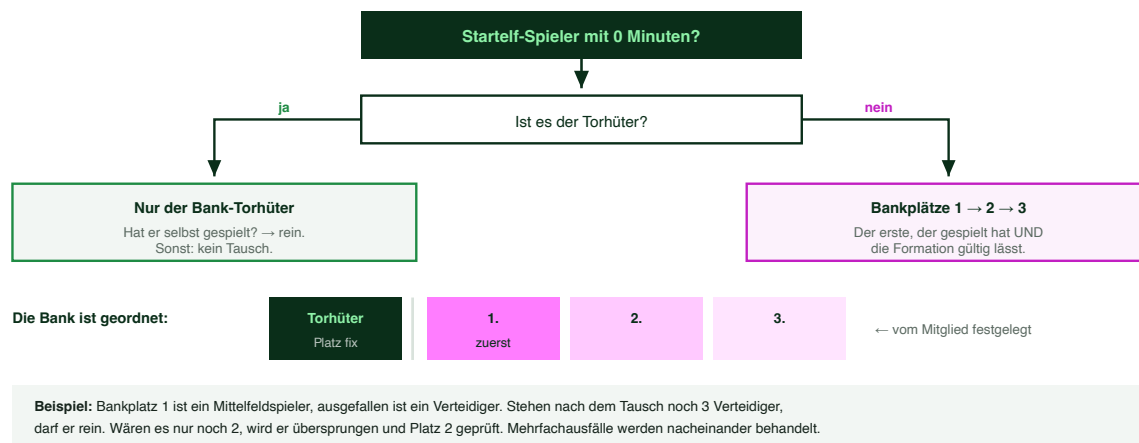
Wer eine Runde lang nichts ändert, soll nicht leer ausgehen. Beim Sperren einer Runde wird für jeden Kader ohne Aufstellung eine Kopie angelegt.

DER FEHLER, DER HIER GEMACHT WURDE

Die erste Fassung kopierte für **jeden** Kader ohne Aufstellung — auch für Neueinsteiger. Wer nach Runde 1 dazukam und ein Team speicherte, bekam beim nächsten Sync rückwirkend eine Runde-1-Aufstellung **samt deren Punkten**. Ein Mitglied erhielt so 73 Punkte für eine Runde, in der es gar nicht dabei war.

Korrekte Regel: Rollover nur für Kader, die an einer *früheren* Runde bereits teilgenommen haben (erkennbar an einer vorhandenen Aufstellung). Kopiert wird aus der letzten echten Aufstellung, nicht aus dem aktuellen Kader.

6.2 Automatische Einwechslung



Entscheidungsbaum der automatischen Einwechslung. Die Logik liegt als reine Funktion vor (`lib/auto-subst.ts`) und ist damit ohne Datenbank prüfbar.

PRÜFSZENARIOEN, DIE EIN NACHBAU BESTEHEN MUSS

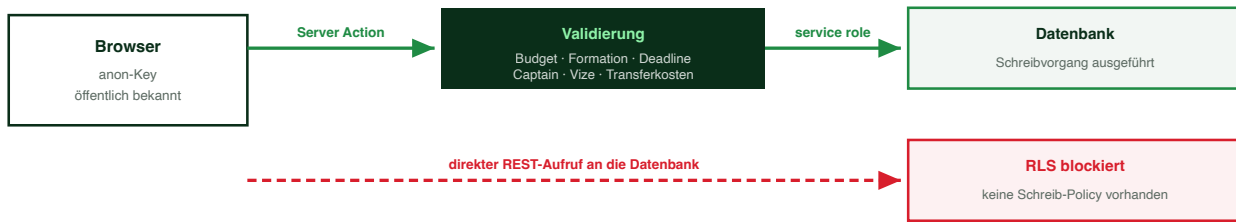
Alle spielen → kein Tausch. Ein Verteidiger fällt aus → Bankplatz 1. Torhüter fällt aus → nur der Bank-Torhüter. Bank-Torhüter spielte auch nicht → kein Tausch. Bei exakt 3 Verteidigern: ein Mittelfeldspieler auf Platz 1 wird übersprungen. Zwei Verteidiger fallen aus, nur einer ist auf der Bank → der zweite wird durch einen Mittelfeldspieler ersetzt, falls die Formation es hergibt. Bankspieler ohne Einsatz werden übersprungen. Drei Ausfälle → alle drei Bankplätze der Reihe nach.

6.3 Bindenübergabe

Spielt der Captain keine Minute und der Vize schon, wandert die Markierung `is_captain` auf den Vize — **bevor** die Einwechslungen laufen. Weil Rangliste, Spieltagspunkte, Feldanzeige und Punkte-Popup alle an derselben Markierung hängen, stimmt danach alles Nachgelagerte von selbst. Es gibt keine zweite Stelle, an der die Verdopplung nachgezogen werden müsste.

7 Sicherheitsmodell

Die Leitfrage lautete: *Was könnte jemand tun, der die Oberfläche umgeht und direkt mit der Datenbank-API spricht?* Der öffentliche anon-Schlüssel steht in jedem Browser-Bundle — er ist kein Geheimnis.



Der Kniff

Für die Spielstandstabellen existiert schlicht keine Schreib-Policy. Ohne passende Policy verweigert Row Level Security jeden Zugriff — der Browser kann also gar nichts schreiben. Der Dienstschlüssel umgeht RLS und wird ausschliesslich serverseitig verwendet, nach Prüfung von Anmeldung und Spielregeln.

Es gibt genau einen Schreibweg: durch die serverseitige Validierung. Der direkte Weg ist nicht abgesichert, sondern schlicht nicht vorhanden.

Ergebnisse des Angriffstests

Mit einer echten Nutzersitzung wurden dreizehn Angriffe gefahren. Zwölf wurden blockiert, einer legte eine echte Lücke offen.

ANGRIFF	ERGEBNIS
Eigene Punkte auf 999 setzen	blockiert
Kader direkt beschreiben (Validierung umgehen)	blockiert
Fremde Aufstellung manipulieren	blockiert
Transferkosten löschen	blockiert
Sich selbst Punkte gutschreiben	blockiert
Spielerpreis senken	blockiert
Sich selbst zum Admin machen	blockiert
Deadline verschieben	blockiert
Chip nachträglich zurücknehmen	blockiert
Rohstatistiken fälschen	blockiert
Fremde E-Mail auslesen	blockiert (nicht in der Tabelle)
Passwort-Login direkt gegen die API	blockiert (Bot-Schutz)
Aktuelle Aufstellung anderer auslesen	gelang — sogar ohne Login

DIE GEFUNDENE LÜCKE

Lesezugriff war mit `using (true)` für alle offen. Die Oberfläche zeigt fremde Teams erst nach der Deadline — die API lieferte sie jedoch sofort. Damit liess sich vor der Deadline die Aufstellung der Konkurrenz abschreiben.

Behoben durch: `squad_players` nur für Besitzer und Admins; `gameweek_squads` zusätzlich für alle, sobald die Deadline der betreffenden Runde abgelaufen ist. Die Rangliste bleibt korrekt, weil sie nur Punkte summiert — und die gibt es ausschliesslich für vergangene Runden.

LEHRE FÜR DEN NACHBAU

Eine Regel, die nur in der Oberfläche durchgesetzt wird, ist keine Regel. Jede Sichtbarkeits- und Schreibbeschränkung muss zusätzlich in der Datenbank stehen — und zwar so, dass sie auch für den nicht angemeldeten Zugriff gilt.

8 Administration

Der laufende Betrieb erfordert im Normalfall **keinen** Eingriff. Die Admin-Werkzeuge dienen der Fehlerkorrektur und der Sichtbarkeit.

WERKZEUG	ZWECK	HÄUFIGKEIT
Sync-Ampel	Zeitstempel des letzten Laufs; rot ab 35 Minuten Stille. Verrät stille Ausfälle, bevor Mitglieder sich beschweren.	passiv
Jetzt synchronisieren	Stösst denselben Endpunkt an wie der Zeitgeber — kein zweiter Codepfad.	bei Bedarf
Ankündigungs-Banner	Freitext, erscheint auf jeder Seite für alle. Leeren blendet aus.	im Störfall
Statistik-Korrektur	Jeder Rohwert je Spieler und Runde überschreibbar, mit Notiz. Danach „Alle Punkte neu berechnen“.	selten
Punkt-Anpassung	Gutschrift oder Abzug mit Begründung — u. a. für Späteinsteiger.	bei Neuzugängen
Spielerverwaltung	Anlegen, umbenennen, Club/Position/Preis ändern, deaktivieren.	Transferfenster
Mitgliederverwaltung	Sperren, entsperren, zum Admin ernennen.	selten
Posteingang	Meldungen aus dem Profil, mit Erledigt-Markierung.	laufend

SCHUTZVORKEHRUNGEN IM ADMIN-BEREICH

Ein Admin kann sich nicht selbst degradieren, der letzte verbleibende Admin ist geschützt, gesperrte Konten lassen sich nicht befördern, und Admin-Konten können sich nicht selbst löschen. Spieler-IDs und die API-Kennung sind im Formular bewusst nicht bearbeitbar — eine falsche Kennung würde stillschweigend falsche Punkte synchronisieren.

Wiederkehrende Aufgaben im Jahresverlauf

WANN	WAS
Vor Saisonstart	Kader aller Clubs prüfen, Preise festlegen. Preisänderungen möglichst vor den ersten gespeicherten Teams — sie wirken sofort auf alle.
Nach jedem Spieltag	Stichprobe: Resultate und beste Spieler plausibel? Bei Bedarf korrigieren.
Winterpause	Zugänge anlegen, Abgänge deaktivieren (nicht löschen).
Bei Neuzugängen	In den ersten 5 Runden: Punkteschnitt gutschreiben.
Datenabo	Laufzeit im Blick behalten — läuft es aus, endet der Sync stumm (die Ampel zeigt es).

9 Fallstricke und Lehren

Diese Fehler wurden beim Bau tatsächlich gemacht und gefunden. Sie sind der wertvollste Teil dieser Dokumentation: Jeder davon war in einem Test unauffällig und wäre im Betrieb teuer geworden.

FEHLER	AUSWIRKUNG	LÖSUNG
Zeitzone	Server rechnet in UTC. Alle Zeiten erschienen zwei Stunden zu früh — die Deadline lag real <i>nach</i> dem Anpfiff.	Jede Datumsausgabe explizit mit <code>timeZone: "Europe/Zurich"</code> .

Gegentore aus dem Spielerfeld	Die API liefert dieses Feld nur für Torhüter zuverlässig; bei Feldspielern kam <code>null</code> und wurde als 0 gelesen — jeder Verteidiger bekam +4 für Zu-Null.	Gegentore aus dem Spielresultat ableiten.
Zwischenspeicher der API	Ein Antwort-Cache von 60 Sekunden lieferte veraltete Daten; zwei von sechs Partien blieben dauerhaft auf „nicht begonnen“.	<code>cache: "no-store"</code> bei allen Live-Abfragen.
Transferzähler	Ein Zählerfeld wurde bei jedem Speichern zurückgesetzt — alle Transfers waren faktisch gratis.	Guthaben aus der Transferhistorie herleiten.
Deadline nur manuell	Ohne Admin-Klick blieb die Runde offen; Teams liessen sich während laufender Spiele ändern.	Automatische Sperrung im Sync und serverseitige Deadline-Prüfung beim Speichern.
Schreibrechte für Besitzer	Wer seinen eigenen Kader direkt beschreiben durfte, konnte die gesamte Validierung umgehen — inklusive selbst gesetzter Punktzahlen.	Alle Schreib-Policies entfernen; ausschliesslich serverseitig schreiben.
Leserechte für alle	Aktuelle Aufstellungen waren über die API abrufbar — auch ohne Login. Abschreiben vor der Deadline war möglich.	Lesen an Besitz und abgelaufene Deadline knüpfen.
Rollover ohne Vorgeschichte	Neueinsteiger erhielten rückwirkend Aufstellungen und Punkte für Runden, in denen sie nicht dabei waren.	Rollover nur bei vorhandener früherer Aufstellung.
Gegentor-Abzug bei Kurzeinsätzen	Ein für 17 Minuten eingewechselter Verteidiger kassierte den vollen Abzug für Tore, die vorher fielen — sein Einsatzpunkt wurde auf 0 aufgefressen.	Abzug erst ab 60 Minuten.
Formation nicht geprüft	Eine gespeicherte Elf enthielt 0 Stürmer.	Geteiltes Regelmodul, von Oberfläche <i>und</i> Server genutzt; zusätzlich eine Machbarkeitsprüfung, die Sackgassen verhindert.
Zeitstempel bei Aktualisierung	Ein Standardwert <code>now()</code> greift nur beim Einfügen, nicht beim Aktualisieren — die Anzeige zeigte dauerhaft den Erstimport.	Zeitstempel ausdrücklich mitschreiben.
Typ-Explosion	Verschachtelte Typen liessen jede Datenbankabfrage stillschweigend zu <code>never</code> werden — Fehlermeldungen zeigten auf völlig andere Stellen.	Zeilentypen flach halten, keine Schnittmengen-Typen.

ÜBERGREIFENDE LEHRE

Sechs der zwölf Fehler betrafen **Regeln, die nur an einer Stelle standen** — entweder nur in der Oberfläche oder nur an einem von zwei Rechenwegen. Was doppelt gilt, muss aus einer gemeinsamen Quelle kommen. Was verbindlich ist, gehört auf den Server. Und was in der Datenbank steht, muss die Datenbank selbst schützen.

10 Anleitung für einen Nachbau

Empfohlene Reihenfolge. Jede Stufe ist für sich lauffähig und prüfbar — das ist wichtiger als Geschwindigkeit, weil Fehler in der Punktelogik später kaum auffallen.

- 1 **Datenmodell und Stammdaten**
Tabellen anlegen, Clubs und Spieler importieren, Spieltage mit Deadlines. Noch keine Oberfläche.
- 2 **Punktelogik als reine Funktion**
Rohdaten hinein, Punkte plus Aufschlüsselung heraus. Gegen die Regeltabelle von Hand prüfen — idealerweise gegen eine unabhängig geschriebene zweite Implementation.
- 3 **Datenanbindung und Zeitgeber**
Sync-Endpunkt mit Sperrlogik. Erst wenn eine vergangene Runde korrekt gerechnet wird, weitergehen.
- 4 **Anmeldung und Rechtemodell**
Row Level Security VOR der Oberfläche einrichten — nachträglich abzudichten ist deutlich mühsamer.
- 5 **Oberfläche**
Team-Aufstellung, Rangliste, Statistiken. Validierung aus demselben Modul wie der Server.
- 6 **Automatiken und Admin-Werkzeuge**
Rollover, Einwechslung, Bindenübergabe, Überwachung. Zuletzt, weil sie das Fundament voraussetzen.

Baureihenfolge. Die Stufen 2 und 4 sind die kritischen — beide lassen sich prüfen, ohne dass eine Oberfläche existiert.

Prüfpunkte vor dem ersten echten Spieltag

PRÜFUNG	
☐	Punkte eines vergangenen Spieltags stimmen mit einer von Hand gerechneten Aufstellung überein — alle 15 Spieler einzeln.
☐	Captain-Verdopplung greift auch bei negativer Punktzahl.
☐	Nach Ablauf der Deadline lässt sich nichts mehr speichern — auch nicht mit veralteter Seite im Browser.
☐	Fremde Aufstellungen sind vor der Deadline nicht abrufbar — auch nicht über die API, auch nicht ohne Login.
☐	Ein direkter Schreibversuch auf Kader und Punkte wird abgewiesen.
☐	Ein Neueinsteiger erhält keine Punkte für Runden vor seinem Beitritt.
☐	Transferkosten werden von der Gesamtpunktzahl abgezogen.
☐	Eine ungültige Formation lässt sich nicht speichern.
☐	Ein vierter Spieler desselben Clubs wird abgewiesen — auch beim Speichern direkt am Server.
☐	Alle angezeigten Zeiten stimmen mit der lokalen Zeitzone überein.
☐	Fällt der Zeitgeber aus, wird das sichtbar.

Was in der Praxis am meisten Zeit gespart hat

- **Regeln an einer Stelle definieren** und von beiden Seiten importieren. Jede zweite Kopie driftet.
- **Reine Funktionen für Spiellogik.** Punkteberechnung und Einwechslung ohne Datenbankzugriff — dadurch in Sekunden gegen Dutzende Szenarien prüfbar statt durch mühsames Simulieren.
- **Werte aus der Datenbank in Regeltextran anzeigen.** Die Regelseite liest Budget und Kadergrößen aus den Einstellungen — sie kann gar nicht veralten.
- **Vor jedem Eingriff sichern, danach vergleichen.** Ein Abzug gegen den Vorher-Stand beweist, dass ein Test nichts hinterlassen hat.
- **Empirisch prüfen statt behaupten.** „Die Transferkosten werden abgezogen“ war so lange falsch, bis es jemand nachgerechnet hat.

AUFWANDSSCHÄTZUNG

Der beschriebene Umfang entspricht rund **8'350 Zeilen** Anwendungscode in 68 Dateien plus 15 Datenbank-Migrationen. Der überwiegende Aufwand lag nicht im Schreiben, sondern im **Finden der zwölf Fallstricke** aus Kapitel 9 — wer sie kennt, kommt deutlich schneller ans Ziel.

Angeli — Swiss League Fantasy · Technische Dokumentation · Stand Juli 2026

Inoffizielles, nicht-kommerzielles Fan-Projekt ohne Verbindung zur Swiss Football League oder deren Clubs.